

💡 Edita o prueba cualquier diagrama en [mermaid.live](https://mermaid.live) — pega el código Mermaid del bloque correspondiente.

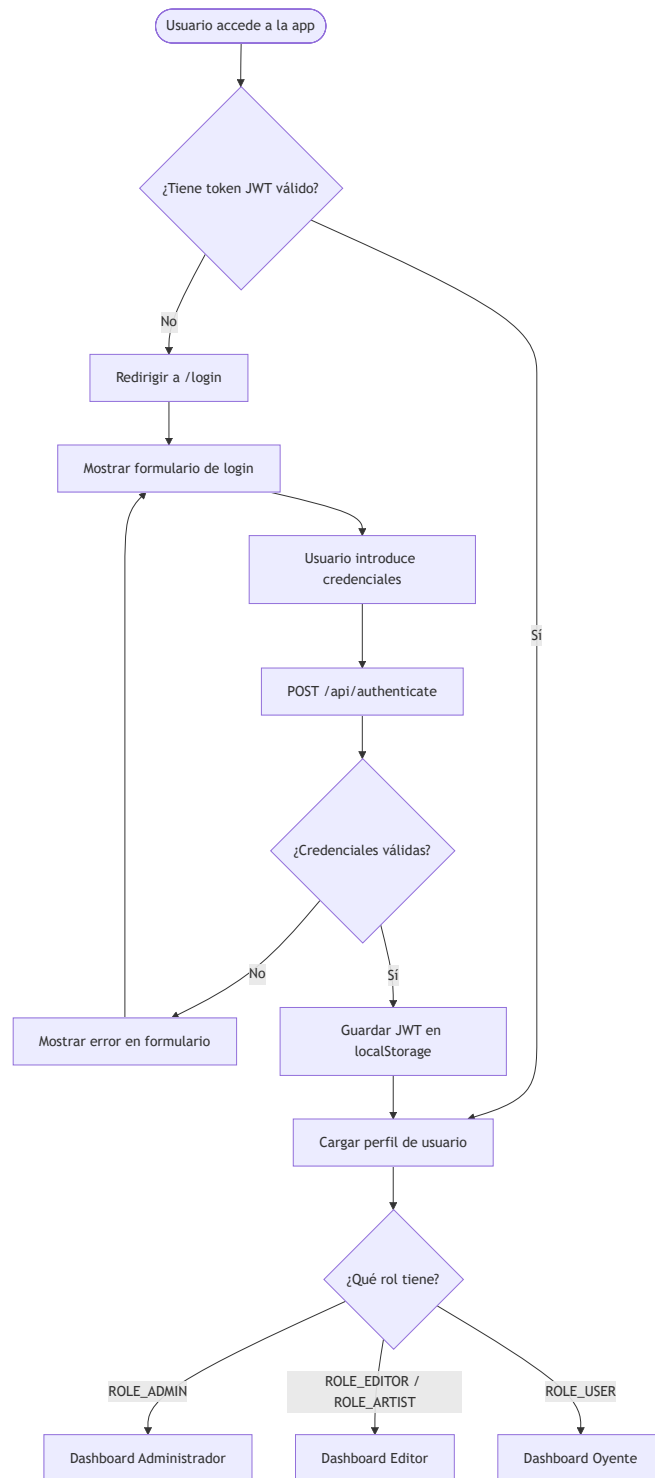
# Diagramas de Flujo — MediaPlayer

Los diagramas están escritos en sintaxis **Mermaid** y se renderizan automáticamente en GitHub, GitLab y VS Code (extensión Mermaid Preview).

---

## 1. Flujo de autenticación

```
flowchart TD
    A([Usuario accede a la app]) --> B{¿Tiene token JWT válido?}
    B -- Sí --> C[Cargar perfil de usuario]
    B -- No --> D[Redirigir a /login]
    D --> E[Mostrar formulario de login]
    E --> F[Usuario introduce credenciales]
    F --> G[POST /api/authenticate]
    G --> H{¿Credenciales válidas?}
    H -- No --> I[Mostrar error en formulario]
    I --> E
    H -- Sí --> J[Guardar JWT en localStorage]
    J --> C
    C --> K{¿Qué rol tiene?}
    K -- ROLE_ADMIN --> L[Dashboard Administrador]
    K -- ROLE_EDITOR / ROLE_ARTIST --> M[Dashboard Editor]
    K -- ROLE_USER --> N[Dashboard Oyente]
```



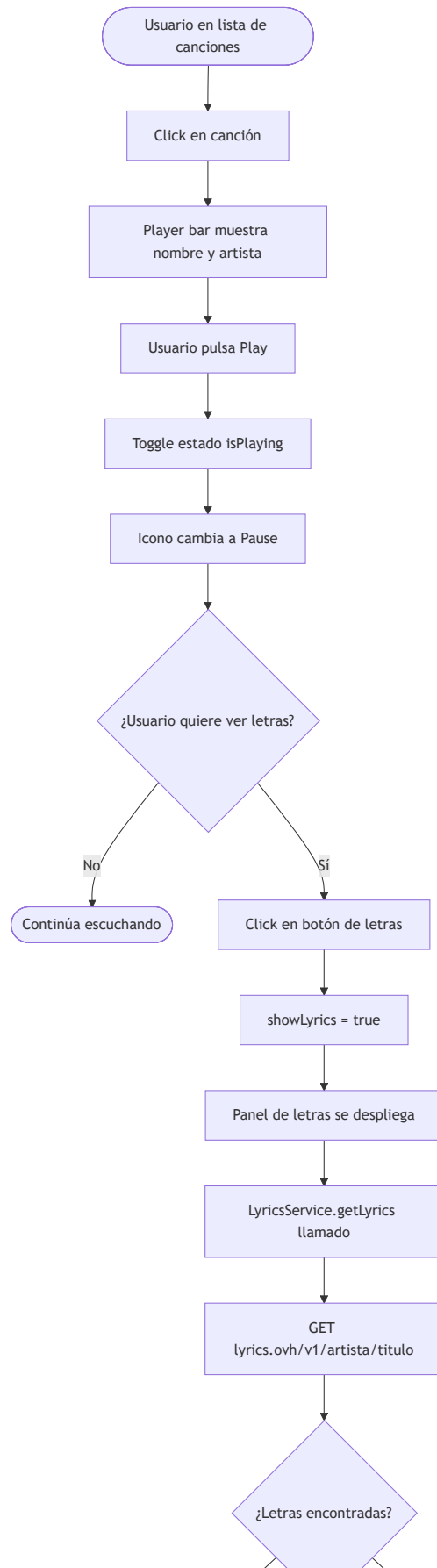
## 2. Flujo de reproducción y letras de canción

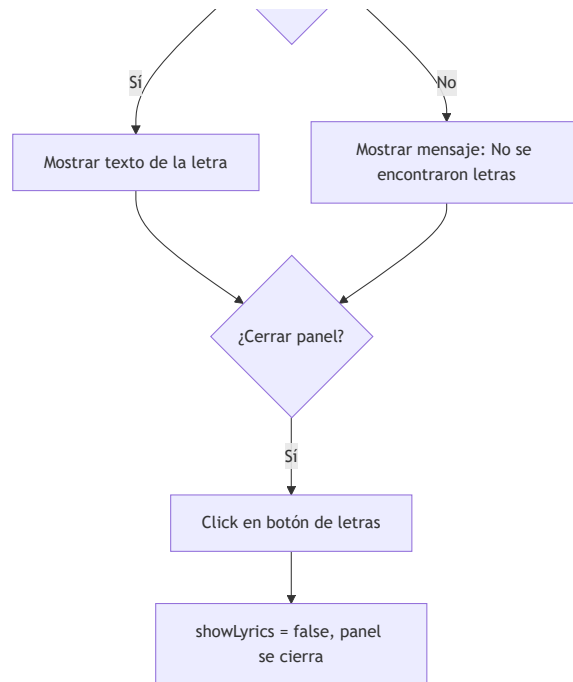
flowchart TD

```

A([Usuario en lista de canciones]) --> B[Click en canción]
B --> C[Player bar muestra nombre y artista]
C --> D[Usuario pulsa Play]
D --> E[Toggle estado isPlaying]
  
```

```
E --> F[Icono cambia a Pause]
F --> G{¿Usuario quiere ver letras?}
G -- No --> H([Continúa escuchando])
G -- Sí --> I[Click en botón de letras]
I --> J[showLyrics = true]
J --> K[Panel de letras se despliega]
K --> L[LyricsService.getLyrics llamado]
L --> M[GET lyrics.ovh/v1/artista/titulo]
M --> N{¿Letras encontradas?}
N -- Sí --> O[Mostrar texto de la letra]
N -- No --> P[Mostrar mensaje: No se encontraron letras]
O --> Q{¿Cerrar panel?}
P --> Q
Q -- Sí --> R[Click en botón de letras]
R --> S[showLyrics = false, panel se cierra]
```



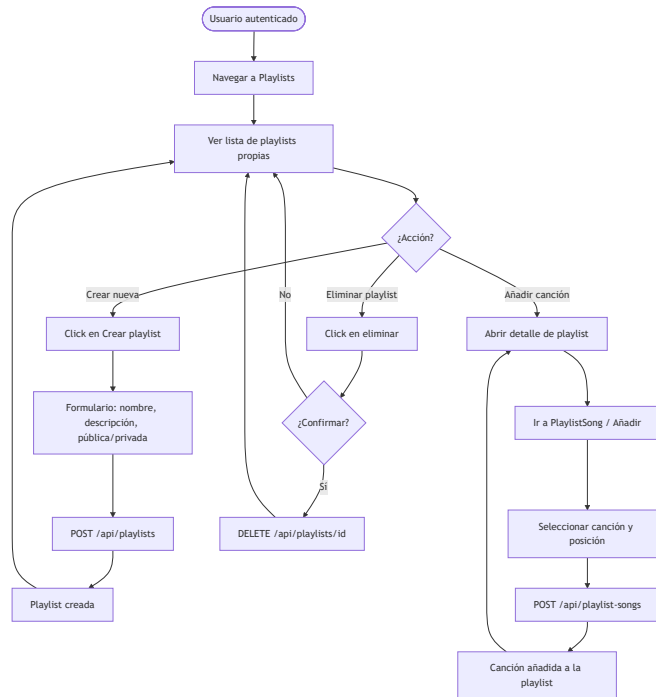


### 3. Flujo de gestión de playlist

flowchart TD

```

A([Usuario autenticado]) --> B[Navegar a Playlists]
B --> C[Ver lista de playlists propias]
C --> D{¿Acción?}
D -- Crear nueva --> E[Click en Crear playlist]
E --> F[Formulario: nombre, descripción, pública/privada]
F --> G[POST /api/playlists]
G --> H[Playlist creada]
H --> C
D -- Añadir canción --> I[Abrir detalle de playlist]
I --> J[Ir a PlaylistSong / Añadir]
J --> K[Seleccionar canción y posición]
K --> L[POST /api/playlist-songs]
L --> M[Canción añadida a la playlist]
M --> I
D -- Eliminar playlist --> N[Click en eliminar]
N --> O{¿Confirmar?}
O -- No --> C
O -- Sí --> P[DELETE /api/playlists/id]
P --> C
  
```

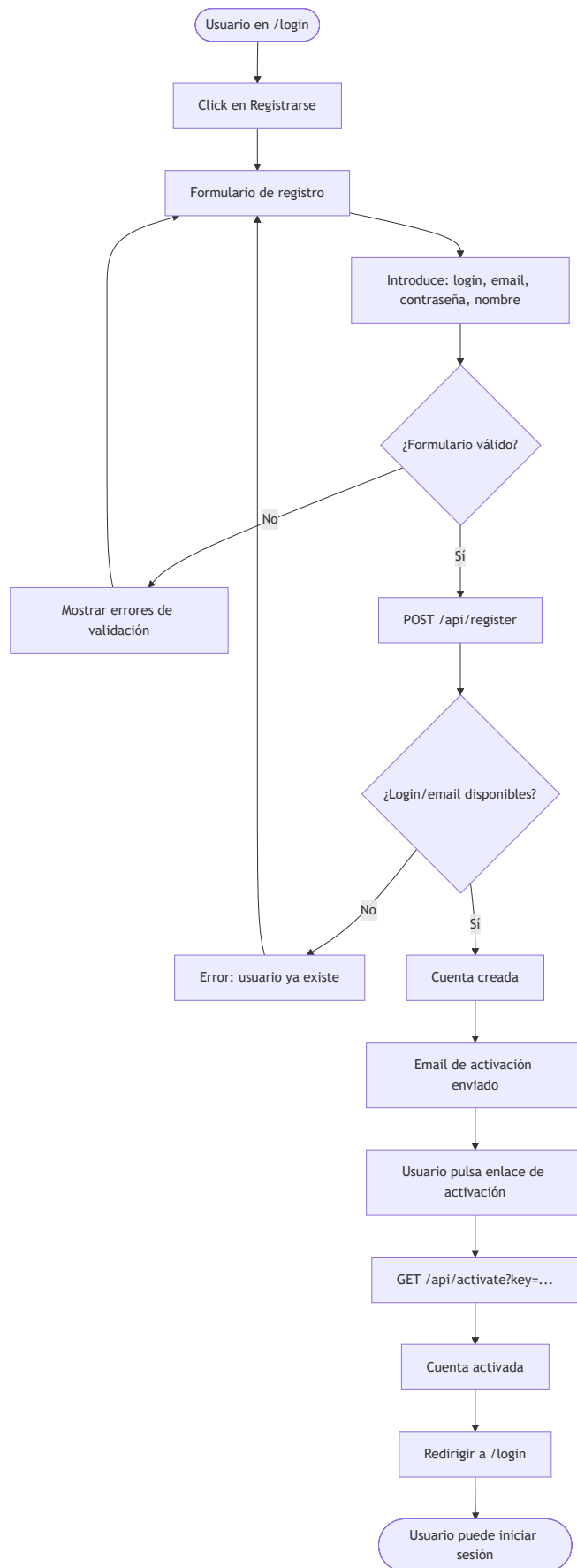


## 4. Flujo de registro de usuario

flowchart TD

```

A([Usuario en /login]) --> B[Click en Registrarse]
B --> C[Formulario de registro]
C --> D[Introduce: login, email, contraseña, nombre]
D --> E{¿Formulario válido?}
E -- No --> F[Mostrar errores de validación]
F --> C
E -- Sí --> G[POST /api/register]
G --> H{¿Login/email disponibles?}
H -- No --> I[Error: usuario ya existe]
I --> C
H -- Sí --> J[Cuenta creada]
J --> K[Email de activación enviado]
K --> L[Usuario pulsa enlace de activación]
L --> M[GET /api/activate?key=...]
M --> N[Cuenta activada]
N --> O[Redirigir a /login]
O --> P([Usuario puede iniciar sesión])
  
```



---

## 5. Diagrama de componentes Angular

```
graph TD
    subgraph AppRoot
        APP[App Component]
        MAIN[Main Component]
    end

    subgraph Layouts
        NAV[Navbar]
        SIDE[Sidebar]
        PLAYER[PlayerBar]
    end

    subgraph Home
        H[HomeComponent]
        DA[DashboardAdmin]
        DE[DashboardEditor]
        DU[DashboardUser]
    end

    subgraph Entities
        SONG[SongComponent]
        ALBUM[AlbumComponent]
        ARTIST[ArtistComponent]
        PLAYLIST[PlaylistComponent]
        GENRE[GenreComponent]
        LIKE[LikeComponent]
        PLAY[PlayComponent]
    end

    subgraph Core
        AUTH[AuthService]
        ACCOUNT[AccountService]
        LYRICS[LyricsService]
        INTERCEPTORS[Interceptores HTTP]
    end

    APP --> MAIN
    MAIN --> NAV
    MAIN --> SIDE
    MAIN --> PLAYER
    MAIN --> H
    H --> DA
    H --> DE
    H --> DU
    MAIN --> SONG
    MAIN --> ALBUM
    MAIN --> ARTIST
    MAIN --> PLAYLIST
    MAIN --> GENRE
    MAIN --> LIKE
    MAIN --> PLAY
    PLAYER --> LYRICS
    NAV --> ACCOUNT
    SIDE --> ACCOUNT
    H --> ACCOUNT
```

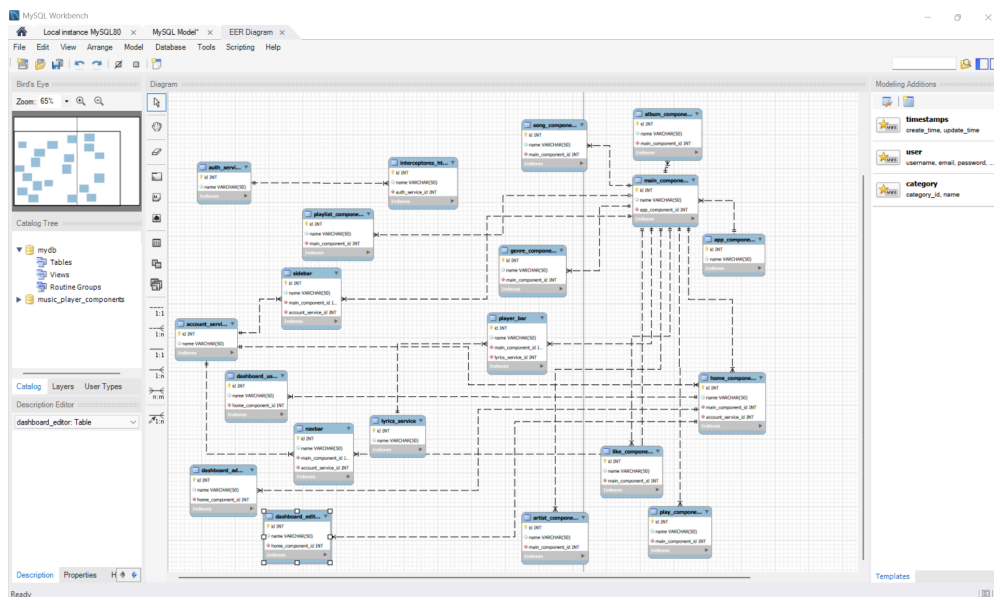


AUTH --> INTERCEPTORS

```
linkStyle 0 stroke:#4285f4,stroke-width:2px
linkStyle 1,2,3 stroke:#34a853,stroke-width:2px
linkStyle 4,5,6,7 stroke:#fbbc04,stroke-width:2px
linkStyle 8,9,10,11,12,13,14 stroke:#9c27b0,stroke-width:2px
linkStyle 15,16,17,18,19 stroke:#ea4335,stroke-width:2px
```



Diagrama MySQL Workbench (Reverse Engineer):



Script MySQL — ejecutar en MySQL Workbench → Database > Reverse Engineer:

```
-- =====
-- Diagrama 5: Arquitectura de Componentes Angular
-- MusicPlayer TFG
-- Uso: ejecutar en MySQL Workbench → luego
--       Database > Reverse Engineer → ver EER Diagram
-- =====
```

```
DROP DATABASE IF EXISTS music_player_components;
CREATE DATABASE music_player_components CHARACTER SET utf8mb4;
USE music_player_components;
```

```
-- — AppRoot —————
CREATE TABLE app_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'AppComponent'
);

CREATE TABLE main_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'MainComponent',
  app_component_id INT NOT NULL,
  FOREIGN KEY (app_component_id) REFERENCES app_component(id)
```

```

);

-- — Core (sin dependencias hacia arriba) —————
CREATE TABLE account_service (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'AccountService'
);

CREATE TABLE auth_service (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'AuthService'
);

CREATE TABLE interceptores_http (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'InterceptoresHTTP',
  auth_service_id INT NOT NULL,
  FOREIGN KEY (auth_service_id) REFERENCES auth_service(id)
);

-- — Layouts —————
CREATE TABLE navbar (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'Navbar',
  main_component_id INT NOT NULL,
  account_service_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id),
  FOREIGN KEY (account_service_id) REFERENCES account_service(id)
);

CREATE TABLE sidebar (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'Sidebar',
  main_component_id INT NOT NULL,
  account_service_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id),
  FOREIGN KEY (account_service_id) REFERENCES account_service(id)
);

CREATE TABLE lyrics_service (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'LyricsService'
);

CREATE TABLE player_bar (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'PlayerBar',
  main_component_id INT NOT NULL,
  lyrics_service_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id),
  FOREIGN KEY (lyrics_service_id) REFERENCES lyrics_service(id)
);

-- — Home —————
CREATE TABLE home_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'HomeComponent',
  main_component_id INT NOT NULL,
  account_service_id INT NOT NULL,

```

```

    FOREIGN KEY (main_component_id) REFERENCES main_component(id),
    FOREIGN KEY (account_service_id) REFERENCES account_service(id)
);

CREATE TABLE dashboard_admin (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) DEFAULT 'DashboardAdmin',
    home_component_id INT NOT NULL,
    FOREIGN KEY (home_component_id) REFERENCES home_component(id)
);

CREATE TABLE dashboard_editor (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) DEFAULT 'DashboardEditor',
    home_component_id INT NOT NULL,
    FOREIGN KEY (home_component_id) REFERENCES home_component(id)
);

CREATE TABLE dashboard_user (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) DEFAULT 'DashboardUser',
    home_component_id INT NOT NULL,
    FOREIGN KEY (home_component_id) REFERENCES home_component(id)
);

-- — Entities —————
CREATE TABLE song_component (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) DEFAULT 'SongComponent',
    main_component_id INT NOT NULL,
    FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

CREATE TABLE album_component (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) DEFAULT 'AlbumComponent',
    main_component_id INT NOT NULL,
    FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

CREATE TABLE artist_component (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) DEFAULT 'ArtistComponent',
    main_component_id INT NOT NULL,
    FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

CREATE TABLE playlist_component (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) DEFAULT 'PlaylistComponent',
    main_component_id INT NOT NULL,
    FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

CREATE TABLE genre_component (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) DEFAULT 'GenreComponent',
    main_component_id INT NOT NULL,
    FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

```

```

);

CREATE TABLE like_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'LikeComponent',
  main_component_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

CREATE TABLE play_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'PlayComponent',
  main_component_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

```

---

## 6. Flujo de control de acceso por rol

flowchart TD

```

A([Petición de navegación]) --> B[AuthGuard.canActivate]
B --> C{¿Token JWT presente?}
C -- No --> D[Redirigir a /login]
C -- Sí --> E{¿Token válido y no expirado?}
E -- No --> D
E -- Sí --> F{¿Ruta requiere rol específico?}
F -- No --> G[Permitir acceso]
F -- Sí --> H{¿Usuario tiene el rol?}
H -- Sí --> G
H -- No --> I[Redirigir a /accessdenied]

```

